

Changes made in pyFC

Tags: [cclouds](#)

Date: 01-02-2025 at 12:15.

Where's pyFC ?

It's in `~/miniconda3/envs/pycupid-env/lib/python3.6/site-packages/pyFC` environment with python `3.6.13`.

Changes

1. Random seed

When generating the clouds, added a parameter `random_seed` to determine whether to use a random seed or not. In the method `.yield_cubes()` in `clouds.py`:

```
# Random seed
if random_seed:
    np.random.seed()

else:
    np.random.seed(123)
```

2. Logging info

Implemented the writing of a `.log` file with some details about the cloud generation process.

The boolean parameter `summary` determines whether the logging file is written or not. It includes some timing info, and parameters.

The final output file looks like this:

```
pyFC-20250202-130319.log
1  2025-02-02 13:03:19 - INFO - pyFC - ===== Parameters =====
2  2025-02-02 13:03:19 - INFO - pyFC - Mesh:          ni = 25, nj = 25, nk = 25
3  2025-02-02 13:03:19 - INFO - pyFC - Wavenumbers: kmin = 4, kmax = 12.0
4  2025-02-02 13:03:19 - INFO - pyFC - Gaussian:    mean = 1.0, sigma = 1.0
5  2025-02-02 13:03:19 - INFO - pyFC - Index:      beta = -1.66667
6  2025-02-02 13:03:19 - INFO - pyFC - Loop:       max_iter = 10, tol = 0.01
7  2025-02-02 13:03:19 - INFO - pyFC - ===== Iterative process =====
8  2025-02-02 13:03:19 - INFO - pyFC - Iter: 1 | Conv: 1.5854e-01 | Time: 0.01 s | CPU: 9.3% | RAM: 67.3%
9  2025-02-02 13:03:20 - INFO - pyFC - Iter: 2 | Conv: 5.1501e-02 | Time: 0.01 s | CPU: 40.0% | RAM: 67.3%
10 2025-02-02 13:03:20 - INFO - pyFC - Iter: 3 | Conv: 2.1907e-02 | Time: 0.01 s | CPU: 40.0% | RAM: 67.3%
11 2025-02-02 13:03:20 - INFO - pyFC - Iter: 4 | Conv: 1.2124e-02 | Time: 0.00 s | CPU: 20.0% | RAM: 67.3%
12 2025-02-02 13:03:20 - INFO - pyFC - Iter: 5 | Conv: 8.5503e-03 | Time: 0.00 s | CPU: 33.3% | RAM: 67.3%
13 2025-02-02 13:03:20 - INFO - pyFC - ===== Summary =====
14 2025-02-02 13:03:20 - INFO - pyFC - Status: SUCCESS
15 2025-02-02 13:03:20 - INFO - pyFC - Total iterations: 5
16 2025-02-02 13:03:20 - INFO - pyFC - Total time: 0.03 s
```

In `pyFC`, we used the modules `logging`, `time` and `psutil` to get and save this information.

First, initialized the logger in the `LogNormalFractalCube.__init__()` method:

```
# Initialize the logger for the class
self.logger = logging.getLogger("pyFC")
```

Then, in `yield_cubes()` which is run by `gen_cube()`, added the formatting and the contents. This was done in this way:

```
# Enable logging if summary is True
if summary:

    # Configure file handler for this logger
    file_handler = logging.FileHandler(log_name)
    formatter = logging.Formatter(
        "%(asctime)s - %(levelname)s - %(name)s - %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S")
    file_handler.setFormatter(formatter)
    self.logger.addHandler(file_handler)
    self.logger.setLevel(logging.INFO)

# Print the parameters
self.logger.info("==== Parameters =====")
self.logger.info(f"Mesh: ni = {self.ni}, nj = {self.nj}, nk = {self.nk}")
self.logger.info(f"Wavenumbers: kmin = {self.kmin}, kmax = {self.kmax}")
self.logger.info(f"Gaussian: mean = {self.mean}, sigma = {self.sigma}")
self.logger.info(f"Index: beta = {self.beta}")
self.logger.info(f"Loop: max_iter = {self.iter_max}, tol = {self.iter_tol}")
```

Also did something similar within the main loop to extract info about each iteration of the process.

3. Empty canvas

Corrected the error of an empty canvas showing up when importing `pyFC`. The problem was in `utils.py` because some functions used `pax = pl.gca()` as a parameter, which *gets the axes of a figure*. The solution was:

```
def function(pax = None):
    if pax is None:
        pax = pl.gca()
    # Rest of the function
```